

Text Embeddings and Vector Representations

A practical educational document for understanding embeddings and their role in semantic search and RAG.

Learning goals

- Understand what an embedding is.
- Understand why text is converted into vectors.
- Understand semantic similarity and cosine similarity.
- Understand embedding models and dimensions.
- Understand how embeddings are used in vector databases and RAG.

Basic idea

A computer does not naturally understand that “car” and “automobile” have related meanings. An embedding model maps text into a numerical vector space where semantically related items can be positioned near each other.

Example

Text	Conceptual relationship
The cat is sleeping.	Similar to “A kitten is asleep.”
The cat is sleeping.	Less similar to “The stock market increased.”

1. What Is an Embedding?

An embedding is a dense numerical representation of an object such as a word, sentence, paragraph, image, or document. For text, an embedding model transforms input text into a fixed-length vector.

For example, a sentence might become a vector with hundreds or thousands of dimensions. The individual numbers usually do not have simple human-readable meanings. Their usefulness comes from the geometry learned by the model.

Sparse vs dense representation

Representation	Characteristics
One-hot / sparse	Large vector, mostly zeros, weak semantic structure.
Dense embedding	Many meaningful continuous values, learned semantic structure.

Why vectors?

- Vectors can be compared mathematically.
- Similarity can be calculated efficiently.
- Large collections can be indexed for fast retrieval.
- Semantic relationships can be captured better than exact keyword matching alone.

Important warning

An embedding is not a database record and it is not an answer. It is a representation used to support comparison, retrieval, clustering, classification, or other tasks.

2. Semantic Similarity

Semantic similarity measures how closely two pieces of content relate in meaning. Embeddings make this measurable by placing related content closer together in vector space.

Keyword search vs semantic search

Query	Keyword search	Semantic search
How do I reset my password?	Looks for overlapping words.	Can retrieve "forgotten password recovery" even with different wording.

Example

Query: "How can I get my money back?" A semantic retriever may consider a passage titled "Refund policy" relevant even though the word "refund" does not occur in the query.

What similarity means

Similarity is model- and metric-dependent. Two vectors being close does not mean the underlying statements are logically equivalent. A retrieval system should therefore return candidate evidence, not blindly treat similarity as proof.

RAG implication

Good embeddings can improve retrieval, but retrieval quality also depends on chunking, metadata, indexing, query formulation, filtering, and reranking.

3. Cosine Similarity and Distance

Cosine similarity compares the angle between two vectors. It is widely used for text embeddings.

$$\text{cosine_similarity}(\mathbf{A}, \mathbf{B}) = (\mathbf{A} \cdot \mathbf{B}) / (\|\mathbf{A}\| \|\mathbf{B}\|)$$

The value is commonly interpreted as high similarity when vectors point in similar directions. Exact ranges and interpretation can vary depending on the implementation and vector normalization.

Simple intuition

- Same direction → high cosine similarity.
- Perpendicular directions → similarity around zero.
- Opposite direction → negative similarity in the standard mathematical definition.

Dot product and Euclidean distance

Vector databases can use different distance or similarity measures. With normalized vectors, dot product and cosine similarity are closely related. Euclidean distance measures straight-line separation and can behave differently when vector magnitudes vary.

Metric	Main intuition
Cosine similarity	Angle / direction similarity
Dot product	Alignment influenced by magnitude
Euclidean distance	Straight-line geometric distance

4. Embedding Models

An embedding model is trained to map input items into a vector space useful for a particular type of similarity or downstream task.

Model considerations

- Language coverage: Does it work well for the languages in your documents?
- Domain: General-purpose models may behave differently on legal, medical, scientific, or code text.
- Dimension: Larger vectors can increase storage and computation.
- Latency: Embedding generation affects ingestion and query speed.
- Maximum input length: Long documents may need chunking before embedding.
- Quality: Evaluate retrieval on your own questions and documents rather than trusting model names alone.

Embedding consistency

In a typical vector-search system, document chunks and user queries should be embedded in a compatible vector space. Changing the embedding model after indexing generally requires re-embedding the stored documents.

Normalization

Some systems normalize vectors before similarity calculations. Whether normalization is appropriate depends on the embedding model and the vector database configuration.

5. Embeddings in a Vector Database

A vector database stores embeddings together with metadata and often the original text or a reference to it.

Field	Example
chunk_id	attention_004
text	Multi-head attention uses several attention heads...
embedding	[0.12, -0.08, ...]
document	attention.pdf
page	4
topic	multi-head attention

Query process

- Embed the user's question.
- Search the vector index for nearest or most similar vectors.
- Return top-k candidate chunks.
- Optionally apply metadata filters or reranking.
- Pass selected chunks to the language model.

Why metadata matters

Metadata can restrict retrieval to a particular document, page range, topic, date, author, or access level. This is especially useful when a RAG system contains many documents.

Top-k

Top-k means the system returns k candidate results. A larger k provides more evidence but can add irrelevant text and increase prompt size. There is no universal best value.

6. Chunking and Embeddings

A whole PDF is usually too large to embed as one item for precise retrieval. RAG systems therefore split documents into chunks.

Common chunking approaches

- Fixed character or token length.
- Recursive splitting using paragraphs and headings.
- Sentence-based splitting.
- Structure-aware splitting using document sections.

Overlap

Chunk overlap repeats a small amount of text between neighboring chunks. It can preserve context that would otherwise be split across boundaries. Too much overlap increases storage and duplicate retrieval.

Bad chunk

A chunk containing half of a definition and the beginning of an unrelated section may be difficult to retrieve and interpret.

Better chunk

A chunk that contains a heading, a complete concept, and enough surrounding explanation is usually easier for both retrieval and generation.

Key principle

Embedding quality cannot fully compensate for poor chunking. Retrieval is a pipeline, and every stage affects the final answer.

7. Embeddings in RAG

End-to-end flow

- Load PDFs.
- Extract text and preserve useful metadata.
- Split text into meaningful chunks.
- Generate embeddings for each chunk.
- Store embeddings and metadata in a vector index.
- Embed the user's query.
- Retrieve relevant chunks.
- Construct a prompt containing the retrieved evidence.
- Generate an answer grounded in that evidence.

Example

User asks: "What is cosine similarity?" The query is embedded. The vector database finds a chunk in embeddings.pdf explaining cosine similarity. That chunk is added to the LLM context, and the LLM produces a natural-language answer.

Common mistakes

- Using different embedding models for stored documents and queries without compatibility.
- Creating chunks that are too large or too small.
- Retrieving many irrelevant chunks just because k is large.
- Assuming the highest similarity score guarantees factual correctness.
- Ignoring document metadata and source tracking.

Final takeaway

Embeddings turn unstructured content into a searchable numerical representation. In RAG, they are the bridge between a user's question and the most relevant pieces of stored knowledge.